

The Application of Deep Learning to Automated Waste Categorisation Using Image Classification

Louis Vidal

20/06/2025

Introduction

The growing global waste crisis presents a significant environmental challenge, with increasing amounts of rubbish overflowing landfills and polluting ecosystems. To address this issue, more efficient and scalable methods of sorting and recycling materials are urgently needed. In response, I am focusing my Extended Project Qualification on developing a machine learning model capable of classifying images of waste in real time using a live camera feed. The system sorts items into six categories: glass, metal, paper, plastic, cardboard, and general trash. I first stumbled upon this concept after reading about how deep learning models were used to classify birds in the wild. I realised that although differentiating between bird pictures might seem complex due to the few distinguishing features that birds have, some models, such as EfficientNet-B0, ResNet152V2 and DenseNet201, achieved an accuracy of 99.51% (Papers With Code 2023). I realised a similar simplified approach could be implemented for environmental challenges. Initially, I planned to build a model using basic image-processing techniques. However, this led to some unusable accuracy; further details will be provided in later sections. Further research soon revealed that CNNs were far more effective for visual recognition tasks. This encouraged me to explore machine learning frameworks such as PyTorch, as implementing a convolutional neural network from scratch at a low level would have been significantly more complex given the scale and time constraint. This automated classification helps facilitate the recycling process by reducing contamination and improving the efficiency of the sorting. Using AI technology provides a faster, more reliable, and easily expandable way to handle classification compared to the old manual methods. Traditional sorting can be time-consuming, prone to mistakes, and costly to maintain. By cutting down on the need for people to be involved in every step, this approach not only saves money but also improves the quality of recycled materials.

This project investigates how an emerging technology, such as artificial intelligence, might enhance the efficacy of waste management systems, eventually increasing recycling sustainability and accessibility. It highlights the important role intelligent automation can play in addressing environmental issues and demonstrates how modern tools may outperform traditional methods in meeting global sustainability goals. AI represents one of the most powerful tools of our time, capable of transforming industries and tackling global issues. This topic

drew my attention because, unlike traditional algorithmic approaches that rely on rigid rules and predefined input structures, machine learning is better suited to producing accurate results even with unseen or unpredictable data. Its ability to learn patterns from data, rather than being explicitly programmed for every scenario, makes it a more flexible and powerful tool for real-world problems like waste classification. For the goal of classifying rubbish that comes in all colours, shapes, and sizes, artificial intelligence seems like the logical fit for this purpose.

Through this project, my goal goes beyond simply building a functioning model; I aim to gain a deeper understanding of how AI systems are developed, trained, and evaluated. Furthermore, I strive to gain a mathematical understanding of how relevant machine learning systems work. As someone relatively new to this field, I anticipate that this project will involve ongoing learning and experimentation to optimise my final product. From selecting the appropriate model architectures to finding the right datasets, and addressing the limitations of evaluation accuracy and computational resources.

Background Research

Before building the model, I carried out background research into machine learning techniques, more specifically those used for image classification. The task of classifying waste images demands a model capable of understanding textural and spatial features within an image. Furthermore, each pixel value can only be interpreted by considering the adjacent pixel values, known as spatial dependence or spatial correlation. Each category often shares visual features such as colour, shape and backgrounds, making it essential for the model to detect subtle local patterns more accurately to differentiate between the categories. I began by researching multi-layer perceptrons (MLPs), a type of feedforward neural network consisting of fully connected layers with nonlinear activation functions such as the ReLU (Rectified Linear Unit)(Nair & Hinton 2010) or the sigmoid activation. This more basic architecture is more commonly used for simpler projects in introductory machine learning. MLPs require input data to be flattened into a one-dimensional vector, thereby removing all two-dimensional features that are critical to classifying the image. This means that multi-layer perceptrons treat neighbouring pixels the same as much more distant pixels, where the relationship might be irrelevant, preventing them from capturing local dependencies, such as edges or textures. As a result, these basic feedforward neural networks do not generalise as well on visual tasks, particularly when faced with images that require more to learn, more intricate patterns and features, especially when classes share similar visual features. Additionally, these models often fail when faced with variation in lighting or orientation. However, for this specific implementation, the MLP can yield moderately promising results because many classes vary noticeably in colour. This allows the MLP to achieve a reasonable accuracy by primarily relying on colour information, even though it cannot learn important features such as edges or textures. For instance, an image of cardboard might have darker grey-brown tones while glass has a lighter green colour. This approach is limited when classes have overlapping colour profiles, such as plastic and glass, or general waste and paper.

For this project, I utilised the Waste Classification Data dataset from Kaggle (asdasdasdas 2022). It contains over 2500 images of various waste materials categorised into six classes. This dataset presents a valuable challenge for image classification models due to its diverse collection of real-world photos captured under different lighting conditions and backgrounds. Using a publicly available dataset like this allowed me to benefit from a well-curated, labelled dataset, saving time on curating and annotating images myself. The dataset's variety also emphasised the need for a model architecture capable of effectively capturing complex visual patterns, enabling it to generalise across differences in texture, colour, and shape.

I identified the limitations of MLPs and began exploring convolutional neural networks, which are specifically designed to handle more complex features and patterns within image data. CNNs preserve spatial hierarchies using local receptive fields, where each neuron processes only a small region of the input image at a time, and shared weights in the form of convolutional filters that capture context around the pixel. These architectural differences allow CNNs to learn more complex visual features while being relatively small compared to MLPs. Due to their proven success across many fields involving computer vision, CNNs have become the standard for image-based classification and were ultimately chosen for my core architecture for my project. Convolutional neural networks use specialised layers called convolutional layers, which apply a small filter iteratively moving it across the input to detect local features. This works because using the values for pixels around the centre of the kernel captures the context of the area. This works better than traditional "vanilla" layers that only process a single pixel at a time, which do not effectively capture the relationship between a pixel and the surrounding region. Following convolution, pooling layers are used to reduce the dimensions by summarising regions (by taking averages or using the maximum value), which helps to reduce the computation and make the model generalise more efficiently, in turn making it more robust to smaller changes or distortions in the input. Together, this ensemble of layers allows the CNN to learn hierarchical feature representation, making it highly effective for image classification.

Planning and Development

The development phase of my project began with the necessary preprocessing of my data. After some research on image input formats for neural networks, I decided to resize each image to a fixed resolution of 64×64 pixels. This resolution meant a balance between keeping enough visual detail for classification and reducing computational complexity, which was especially important given the hardware limitations I had. A specific algorithm had to be developed to traverse through each subdirectory, treating each subfolder as a separate class label. Initially, each image was loaded with a greyscale value to reduce input size. To make the data compatible with a multilayer perceptron, each image was then flattened into a one-dimensional vector of 4032 values (one for each pixel). This transformation allowed the images to be treated as standard numerical input, but it also meant losing the spatial

structure of the original images, as discussed in the previous section. This limitation, I realised only later in the project. Finally, each pixel value is normalised by dividing by 255 using min-max normalisation. This procedure is used to scale pixel values to a standard range (typically 0 - 1), which helps neural networks to function more efficiently. Without this necessary process, raw pixel values ranging from 0 to 255 can cause large gradients or unstable weights when updating them through linear regression. By compressing the values, the model converges faster relative to denormalised data and becomes less sensitive to differences in input scales.

The general formula for min-max normalisation is as follows:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

At this point, I built a deeper MLP architecture to address these limitations. The final model had four hidden layers with 1028, 512, 256, and 128 neurons, respectively, and an input layer of size 4032 (for the flattened RGB images). The output layer matched the number of final categories: 6. I trained this architecture using 50 epochs, a learning rate of 0.0005, and a batch size of 32. This lower learning rate was used so that the model could converge more smoothly as the model size increased. Despite these changes, the accuracy still plateaued at around 32%. This usually indicates limited generalisation.

The next phase of my project involved an iterative and practical approach to improving the model's accuracy while deepening my mathematical understanding of neural networks. Committed to building a neural network from scratch, I researched the best ways of structuring my code for efficiency and ease of understanding. This led me to adopt an object-oriented approach to structure my model, which allowed me to modularise each component of the layers, such as activations, the training loop and the final prediction logic, promoting code reuse and simplicity. I used numerous tutorials and articles to build my understanding; however, it was the detailed tutorial by EL Caiseri (Caiseri 2022) that served as my key reference throughout this process. This tutorial offered a clear and practical framework for implementing an MLP using NumPy while also deepening my understanding of the mathematical principles behind each component. I followed a modular approach as suggested in the tutorial.

Building on this, I implemented an MLP from scratch to better comprehend how data flows through a neural network. To aid understanding, it's helpful to briefly outline how this architecture is structured. An MLP theoretically acts as a universal approximator, capable of modelling any continuous function given sufficient complexity and training time. A multi-layer perceptron consists of an input layer, one or more hidden layers and an output layer. Each layer of nodes is connected to every node of the next layer, forming a familiar net that you might have seen before.

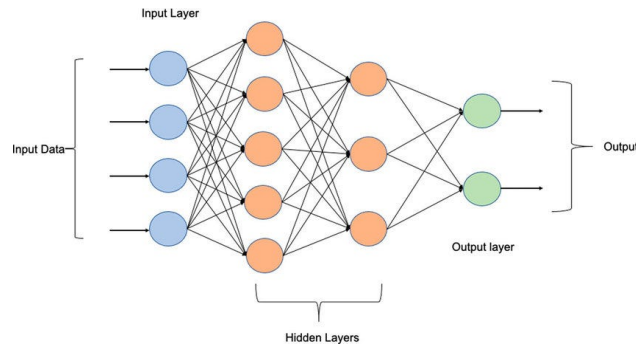


Figure 1: MLP simplified Diagram

Each node in the input layer holds a pixel value from our images, meaning the size of the input layer is determined by the cardinality of the input data. In this case, the size of the input layer is 4032. Every connection between neurons in adjacent layers corresponds to a learnable weight, which determines how strongly each input influences the next layer during forward propagation. I used the ReLU (Rectified Linear Unit) activation function in the hidden layers because it introduces non-linearity, enabling the neural network to learn and represent much more complex patterns than would be possible with solely linear layers.

Unlike sigmoid or tanh activations, which squash output into a limited range and can suffer from vanishing gradients, ReLU outputs zero for negative results and a linear response for positive results. This can be modelled by the max function, $\max(0, x)$. This characteristic often improves convergence by a factor of 6 in deep learning models. However, ReLU may cause some neurons to remain inactive if they output zero consistently.

The next important step is forward propagation, where input data is passed through the network layer by layer. Next, during forward propagation, inputs are multiplied by weights and summed with biases, and passed through an activation function to produce an output. This transforms raw pixel data into predictions. Following this, backpropagation calculates the error between predictions and true labels, then works through the network to update the weights using calculus. This process allows the MLP to learn from its output and iteratively modify its weights to produce a better output.

The MLP class I developed contains three key components: initialisation, forward propagation, and backpropagation. This implementation gave me hands-on insight into how neural networks learn from data, and it prepared me for the more complex CNN model I would later build using PyTorch(PyTorch Contributors 2019).

Model Evaluation and Reflection

The dataset images are divided into two parts, 20% for the testing of the model and 80% for the training process, as evaluating a model with data it has trained on would show inaccurately high results. After completing the MLP implementation, I evaluated its performance

on the waste classification task with the test data. This yielded an unexpectedly low result that was comparable to randomly guessing, revealing the model's inability to capture complex patterns in the data. To improve the model's accuracy, I made several iterative adjustments to the structure of the model. I first assumed that the model wasn't large enough, meaning there were not enough hidden layers to capture more complex features of the images. To account for this, I first increased the number of hidden layers and expanded each layer to include more neurons. This gave the model a greater learning capacity and produced a slight improvement, but not enough to be considered useful. Next, I increased the image resolution to 128×128 pixels to retain more visual detail. Finally, I introduced colour images instead of greyscale, allowing the model to make use of differences in RGB channels. These changes collectively raised the model's accuracy to a maximum of 40%.

Despite this improvement, a further increase in model size led to signs of overfitting. Training accuracy kept increasing over the 80% mark, while validation accuracy plateaued, implying that the model was memorising the training data without generalising effectively. This was a turning point for me where I realised the importance of balancing model complexity with generalisation to avoid unnecessary expansion of architecture where there might be overfitting. Realising the limitations of the MLP, I transitioned to implementing a convolutional neural network (CNN) using the Pytorch framework. CNNs are better suited for classifying image data, as discussed earlier. Pytorch provided a flexible framework for designing and training the CNN with built-in support for CUDA drivers. This enables me to leverage an Nvidia graphics processing unit for significantly faster training. This shift marked a significant step forward in the project, allowing for better accuracy and scalability in the waste classification task.

To assess the impact of these changes, I tracked several performance metrics to better understand how factors like input size or the number of hidden layers affected the model. Specifically, I measured the F1-score, recall, and precision to evaluate performance beyond overall accuracy. Each score measures a different aspect of the model's output to give a more complete representation of the performance. The accuracy of the model can be calculated with the following equation:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positives):** Correctly predicted positive cases.
- **TN (True Negatives):** Correctly predicted negative cases.
- **FP (False Positives):** Incorrectly predicted as positive.
- **FN (False Negatives):** Incorrectly predicted as negative.

Accuracy measures the proportion of correct predictions. However, it can be misleading on imbalanced datasets, as a model may appear accurate simply by predicting the majority class. In such cases, other metrics such as precision or recall may provide more insight.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision reflects the quality of positive predictions. It measures the proportion of true positive predictions among all positive predictions made by the model.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall, or sensitivity, indicates how well the model identifies actual positive cases. In applications like waste classification, high recall ensures important categories are not missed.

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score is the harmonic mean of precision and recall. It provides a balanced metric that is especially useful when both false positives and false negatives carry significant consequences (Kashyap 2024).

After running my CNN model with PyTorch, these are the final scores that were collected.

Table 1: Classification Report by Class

Class	Precision	Recall	F1-score	Support
Cardboard	0.93	0.78	0.85	81
Glass	0.59	0.83	0.69	100
Metal	0.67	0.49	0.56	82
Paper	0.70	0.94	0.80	100
Plastic	0.77	0.60	0.67	97
Misc. Trash	1.00	0.33	0.50	27
Accuracy	0.71 (on 487 samples)			
Macro avg	0.78	0.66	0.68	487
Weighted avg	0.74	0.71	0.70	487

Although the model performs reasonably across most classes, cardboard stands out as the most accurately classified category, achieving a precision of 0.93, a recall of 0.78, and the highest F1-score of 0.85. This suggests that the model consistently identifies cardboard correctly, both in terms of avoiding false positives and capturing most true positives.

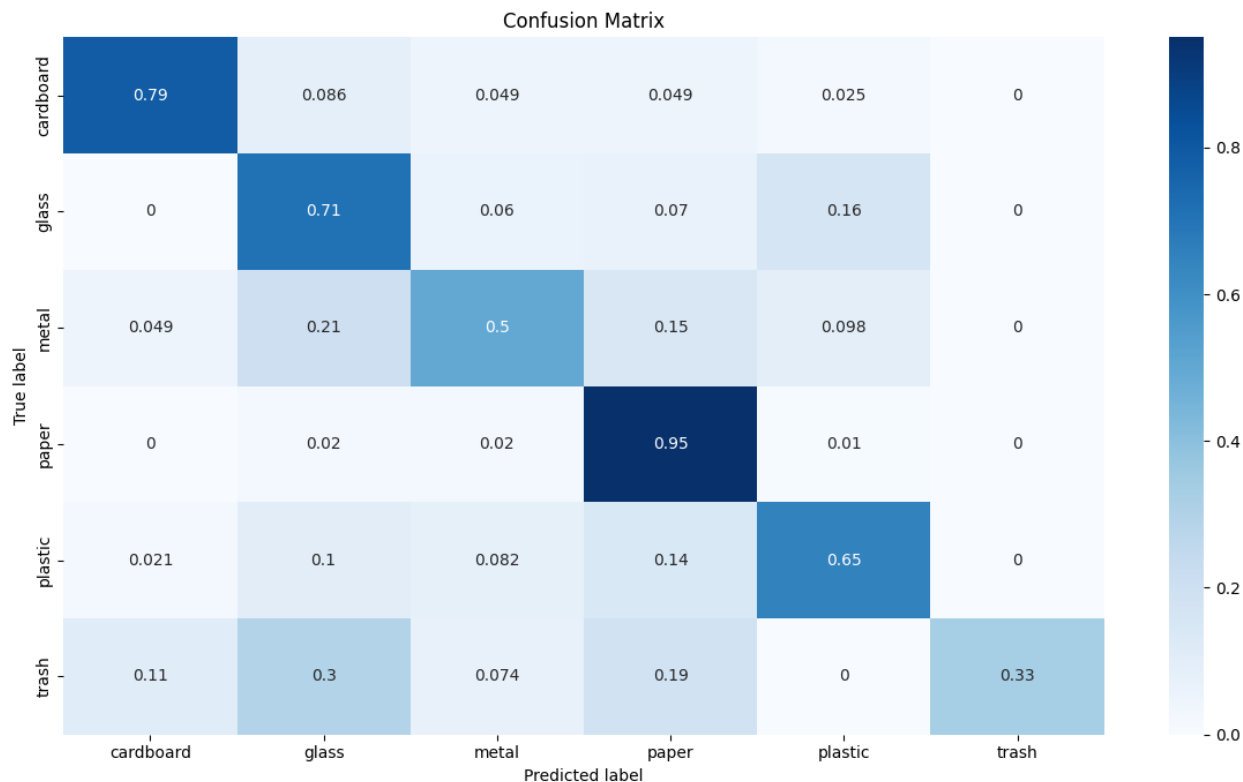


Figure 2: Confusion matrix for CNN

Finally, this is the confusion matrix showing how the model behaved when predicting data. As seen from Figure 2, paper and cardboard stand out as the best recognised categories because they both have sharp edges and uniform colour, meaning it is easier to learn the spatial features, leading to better accuracy.

Conclusion

Throughout the course of this project, I have deepened my mathematical understanding of how Artificial Intelligence works and more specifically how convolutional neural networks can be applied to solve real-world classification tasks. I started this project with limited knowledge on machine learning and I finished with a working image classification model that was able to sort rubbish to a reasonable degree of accuracy.

While the model does very well when it comes to predicting pictures of paper or cardboard, it struggles quite considerably when it comes to handling images with more ambiguous patterns or visually similar items such as plastic and glass. One possible improvement that I could implement would be incorporating augmentation techniques such as rotators, flips, and brightness shift to further increase the robustness of the model. Another way of improving model accuracy going forward would be to transfer learning using pre-trained models such as ResNet or MobileNet.

Looking ahead, I hope to expand this project by integrating the model with real-time hardware using a Raspberry Pi connected to a camera system to allow automatic sorting in a physical environment. I hope to continue learning about the latest developments in machine learning and contribute to sustainable innovation in the future.

References

- asdasdasdas (2022), ‘Waste classification data’, <https://www.kaggle.com/datasets/asdasdasdas/garbage-classification>.
- Caiseri, E. (2022), ‘Building a multi-layer perceptron from scratch with numpy’, <https://elcaiseri.medium.com/building-a-multi-layer-perceptron-from-scratch-with-numpy-e4cee82ab06d>.
- Kashyap, P. (2024), ‘Understanding precision, recall, and f1 score metrics’, <https://medium.com/@piyushkashyap045/understanding-precision-recall-and-f1-score-metrics-ea219b908093>.
- Nair, V. & Hinton, G. E. (2010), Rectified linear units improve restricted boltzmann machines, *in* ‘Proceedings of the 27th International Conference on Machine Learning (ICML-10)’, pp. 807–814.
- Papers With Code (2023), ‘Image classification on cifar-10’, <https://paperswithcode.com/sota/image-classification-on-cifar-10>.
- PyTorch Contributors (2019), ‘Pytorch: An imperative style, high-performance deep learning library’, <https://pytorch.org/>.

In this notebook, we develop a Convolutional Neural Network (CNN) to classify images of waste into six categories: glass, metal, paper, plastic, cardboard, and general trash.

We first need to import the necessary libraries to create the convolutional neural network using pytorch.

```
In [4]: import os
import cv2
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset, random_split
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from sklearn.metrics import confusion_matrix
import seaborn as sn
import matplotlib.pyplot as plt
import random
import pandas as pd
```

```
In [5]: import torch
print("CUDA available:", torch.cuda.is_available())
print("CUDA device name:", torch.cuda.get_device_name(0) if torch.cuda.is_av
```

CUDA available: True

CUDA device name: NVIDIA GeForce RTX 4060 Ti

Now that we have the relevant imports, we can create a garbage dataset class that contains information such as labels and images. In addition, the class should contain a method for returning the length and for fetching information about a specific item.

```
In [ ]: def load_and_preprocess_data(data_dir, img_size=(512, 512)):
    X = []
    y = []
    categories = []

    for d in os.listdir(data_dir):
        if os.path.isdir(os.path.join(data_dir, d)) and not d.startswith('.'):
            categories.append(d)

    print("Found categories:", categories)
    categories.sort()

    print("Loading and preprocessing images...")
    for category_idx, category in enumerate(categories):
        category_path = os.path.join(data_dir, category)
        if not os.path.isdir(category_path):
            continue
```

```

print(f"Processing Category: {category} ({category_idx})")
image_files = [f for f in os.listdir(category_path) if f.lower().endswith('.jpg')]

for img_file in tqdm(image_files[:500]):
    img_path = os.path.join(category_path, img_file)
    img = cv2.imread(img_path)
    img = cv2.resize(img, img_size)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_rgb = img_rgb / 255.0

    X.append(img_rgb)
    y.append(category_idx)

return np.array(X), np.array(y), categories

```

This function loads and preprocesses image data from a specified directory. In this case each subdirectory represents a category. It reads up to 500 ".jpg" images per category, resizing them to 512x512 pixels converting them to RGB and normalizes pixel values (between 0 and 1). The dimensions of the input can be modified.

Now we define the CNN Model.

```

In [7]: class CNN(nn.Module):
    def __init__(self, num_classes):
        super(CNN, self).__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.1),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.1),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout2d(0.3),

            nn.Conv2d(256, 512, kernel_size=3, padding=1),
            nn.BatchNorm2d(512),

```

```

        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Dropout2d(0.3),
    )

    self.flatten = nn.Flatten()
    self.fc1 = None
    self.relu = nn.ReLU()
    self.dropout = nn.Dropout(0.3)
    self.fc2 = None
    self.num_classes = num_classes
    self.initialized = False

    def _initialize_fc_layers(self, x):
        with torch.no_grad():
            x = self.conv_layers(x)
            flat_size = self.flatten(x).size(1)

            self.fc1 = nn.Linear(flat_size, 512).to(x.device)
            self.fc2 = nn.Linear(512, self.num_classes).to(x.device)
            self.initialized = True

    def forward(self, x):
        if not self.initialized:
            self._initialize_fc_layers(x)

        x = self.conv_layers(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

```

This class is a Pytorch model with three convolutional layers for features extraction and two fully connected layers for classifications. It uses Relu activations (max(0,n)), max pooling, and dropout for regularization. The final layers outputs class predictions. Dropout for regularization works by randomly setting a fraction of the neurons to zero to prevent regularisation.

Now let's train the model:

```

In [8]: def train_model(model, train_loader, val_loader, num_epochs=10, learning_rate=1e-4,
        criterion = nn.CrossEntropyLoss(), optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4):

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)

```

```

        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {total_loss/len(train_loader)}")

    evaluate_model(model, val_loader, verbose=True)

```

This function trains the CNN model using the training data over a specified number of epochs. It uses the Adam optimizer and cross-entropy loss to update the model's weights based on prediction error. After each epoch, it prints the average training loss and evaluates the model's performance on the validation set. This helps monitor learning progress and prevent overfitting. Overfitting happens when our model is overadapted to its training data and doesn't perform well on external data.

```

In [9]: def evaluate_model(model, loader, verbose=False):
        model.eval()
        y_true = []
        y_pred = []

        with torch.no_grad():
            for images, labels in loader:
                images = images.to(device)
                outputs = model(images)
                _, predicted = torch.max(outputs.data, 1)
                y_true.extend(labels.numpy())
                y_pred.extend(predicted.cpu().numpy())

        if verbose:
            print("Accuracy:", accuracy_score(y_true, y_pred))
            print("Classification Report:\n", classification_report(y_true, y_pr

```

This function evaluates the model on a dataset and returns prediction results. If "verbose" is "true", it prints the accuracy and a detailed classification report.

```

In [10]: DATA_DIR = "./Garbage_classification"
        X, y, categories = load_and_preprocess_data(DATA_DIR)
        print("Data shape:", X.shape, y.shape)
        num_classes = len(categories)

        for class_idx in range(len(categories)):
            img = X[y == class_idx][random.randint(0, len(X[y == class_idx])-1)]
            plt.imshow(img)
            plt.title(f"Class: {categories[class_idx]}")
            plt.axis('off')
            plt.show()

```

Found categories: ['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']

Loading and preprocessing images...

Processing Category: cardboard (0)

100%|██████████| 403/403 [00:00<00:00, 477.45it/s]

100%|██████████| 403/403 [00:00<00:00, 477.45it/s]

Processing Category: glass (1)

100%|██████████| 500/500 [00:01<00:00, 476.07it/s]

100%|██████████| 500/500 [00:01<00:00, 476.07it/s]

Processing Category: metal (2)

100%|██████████| 410/410 [00:00<00:00, 460.47it/s]

100%|██████████| 410/410 [00:00<00:00, 460.47it/s]

Processing Category: paper (3)

100%|██████████| 500/500 [00:01<00:00, 402.72it/s]

Processing Category: plastic (4)

100%|██████████| 482/482 [00:01<00:00, 389.26it/s]

100%|██████████| 482/482 [00:01<00:00, 389.26it/s]

Processing Category: trash (5)

100%|██████████| 137/137 [00:00<00:00, 398.47it/s]

Data shape: (2432, 512, 512, 3) (2432,)

Class: cardboard



Class: glass



Class: metal



Class: paper



Class: plastic



Class: trash



```
In [11]: # Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, strat
X_train_tensor = torch.tensor(X_train.transpose(0, 3, 1, 2), dtype=torch.flo
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_val_tensor = torch.tensor(X_val.transpose(0, 3, 1, 2), dtype=torch.float32
y_val_tensor = torch.tensor(y_val, dtype=torch.long)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNN(num_classes=num_classes).to(device)

train_model(model, train_loader, val_loader, num_epochs=20)

torch.save(model.state_dict(), "cnn_model.pth")
```

Epoch [1/20], Loss: 1.6639
Accuracy: 0.4702258726899384
Classification Report:

	precision	recall	f1-score	support
0	0.77	0.67	0.72	81
1	0.36	0.58	0.45	100
2	0.64	0.09	0.15	82
3	0.69	0.38	0.49	100
4	0.38	0.74	0.50	97
5	0.00	0.00	0.00	27
accuracy			0.47	487
macro avg	0.47	0.41	0.38	487
weighted avg	0.53	0.47	0.44	487

Accuracy: 0.4702258726899384
Classification Report:

	precision	recall	f1-score	support
0	0.77	0.67	0.72	81
1	0.36	0.58	0.45	100
2	0.64	0.09	0.15	82
3	0.69	0.38	0.49	100
4	0.38	0.74	0.50	97
5	0.00	0.00	0.00	27
accuracy			0.47	487
macro avg	0.47	0.41	0.38	487
weighted avg	0.53	0.47	0.44	487

Epoch [2/20], Loss: 1.4612
Epoch [2/20], Loss: 1.4612
Accuracy: 0.5359342915811088
Classification Report:

	precision	recall	f1-score	support
0	0.71	0.79	0.75	81
1	0.42	0.65	0.51	100
2	0.63	0.21	0.31	82
3	0.73	0.51	0.60	100
4	0.45	0.66	0.53	97
5	0.00	0.00	0.00	27
accuracy			0.54	487
macro avg	0.49	0.47	0.45	487
weighted avg	0.55	0.54	0.51	487

Accuracy: 0.5359342915811088
Classification Report:

	precision	recall	f1-score	support
0	0.71	0.79	0.75	81
1	0.42	0.65	0.51	100
2	0.63	0.21	0.31	82
3	0.73	0.51	0.60	100

4	0.45	0.66	0.53	97
5	0.00	0.00	0.00	27
accuracy			0.54	487
macro avg	0.49	0.47	0.45	487
weighted avg	0.55	0.54	0.51	487

Epoch [3/20], Loss: 1.3623
Epoch [3/20], Loss: 1.3623
Accuracy: 0.5462012320328542
Classification Report:

	precision	recall	f1-score	support
0	0.78	0.74	0.76	81
1	0.46	0.63	0.53	100
2	0.62	0.22	0.32	82
3	0.74	0.57	0.64	100
4	0.41	0.70	0.52	97
5	0.00	0.00	0.00	27
accuracy			0.55	487
macro avg	0.50	0.48	0.46	487
weighted avg	0.56	0.55	0.53	487

Accuracy: 0.5462012320328542
Classification Report:

	precision	recall	f1-score	support
0	0.78	0.74	0.76	81
1	0.46	0.63	0.53	100
2	0.62	0.22	0.32	82
3	0.74	0.57	0.64	100
4	0.41	0.70	0.52	97
5	0.00	0.00	0.00	27
accuracy			0.55	487
macro avg	0.50	0.48	0.46	487
weighted avg	0.56	0.55	0.53	487

Epoch [4/20], Loss: 1.2966
Epoch [4/20], Loss: 1.2966
Accuracy: 0.5790554414784395
Classification Report:

	precision	recall	f1-score	support
0	0.79	0.75	0.77	81
1	0.40	0.70	0.51	100
2	0.68	0.28	0.40	82
3	0.80	0.66	0.72	100
4	0.52	0.64	0.57	97
5	0.00	0.00	0.00	27
accuracy			0.58	487
macro avg	0.53	0.51	0.50	487
weighted avg	0.60	0.58	0.56	487

Accuracy: 0.5790554414784395

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.75	0.77	81
1	0.40	0.70	0.51	100
2	0.68	0.28	0.40	82
3	0.80	0.66	0.72	100
4	0.52	0.64	0.57	97
5	0.00	0.00	0.00	27
accuracy			0.58	487
macro avg	0.53	0.51	0.50	487
weighted avg	0.60	0.58	0.56	487

Epoch [5/20], Loss: 1.2632

Epoch [5/20], Loss: 1.2632

Accuracy: 0.6324435318275154

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.74	0.76	81
1	0.45	0.75	0.56	100
2	0.68	0.30	0.42	82
3	0.84	0.87	0.85	100
4	0.60	0.63	0.61	97
5	0.00	0.00	0.00	27
accuracy			0.63	487
macro avg	0.56	0.55	0.54	487
weighted avg	0.63	0.63	0.61	487

Accuracy: 0.6324435318275154

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.74	0.76	81
1	0.45	0.75	0.56	100
2	0.68	0.30	0.42	82
3	0.84	0.87	0.85	100
4	0.60	0.63	0.61	97
5	0.00	0.00	0.00	27
accuracy			0.63	487
macro avg	0.56	0.55	0.54	487
weighted avg	0.63	0.63	0.61	487

Epoch [6/20], Loss: 1.2218

Epoch [6/20], Loss: 1.2218

Accuracy: 0.6509240246406571

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.77	0.79	81
1	0.53	0.69	0.60	100
2	0.68	0.39	0.50	82

3	0.73	0.88	0.80	100
4	0.59	0.67	0.63	97
5	0.50	0.04	0.07	27
accuracy			0.65	487
macro avg	0.64	0.57	0.56	487
weighted avg	0.65	0.65	0.63	487

Accuracy: 0.6509240246406571

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.77	0.79	81
1	0.53	0.69	0.60	100
2	0.68	0.39	0.50	82
3	0.73	0.88	0.80	100
4	0.59	0.67	0.63	97
5	0.50	0.04	0.07	27
accuracy			0.65	487
macro avg	0.64	0.57	0.56	487
weighted avg	0.65	0.65	0.63	487

Epoch [7/20], Loss: 1.1963

Epoch [7/20], Loss: 1.1963

Accuracy: 0.5913757700205339

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.77	0.81	81
1	0.46	0.76	0.57	100
2	0.49	0.49	0.49	82
3	0.92	0.46	0.61	100
4	0.56	0.66	0.60	97
5	0.00	0.00	0.00	27
accuracy			0.59	487
macro avg	0.55	0.52	0.51	487
weighted avg	0.62	0.59	0.58	487

Accuracy: 0.5913757700205339

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.77	0.81	81
1	0.46	0.76	0.57	100
2	0.49	0.49	0.49	82
3	0.92	0.46	0.61	100
4	0.56	0.66	0.60	97
5	0.00	0.00	0.00	27
accuracy			0.59	487
macro avg	0.55	0.52	0.51	487
weighted avg	0.62	0.59	0.58	487

Epoch [8/20], Loss: 1.1624

Epoch [8/20], Loss: 1.1624
Accuracy: 0.6570841889117043
Classification Report:

	precision	recall	f1-score	support
0	0.84	0.78	0.81	81
1	0.51	0.70	0.59	100
2	0.59	0.44	0.50	82
3	0.79	0.84	0.82	100
4	0.61	0.65	0.63	97
5	0.80	0.15	0.25	27
accuracy			0.66	487
macro avg	0.69	0.59	0.60	487
weighted avg	0.67	0.66	0.65	487

Accuracy: 0.6570841889117043
Classification Report:

	precision	recall	f1-score	support
0	0.84	0.78	0.81	81
1	0.51	0.70	0.59	100
2	0.59	0.44	0.50	82
3	0.79	0.84	0.82	100
4	0.61	0.65	0.63	97
5	0.80	0.15	0.25	27
accuracy			0.66	487
macro avg	0.69	0.59	0.60	487
weighted avg	0.67	0.66	0.65	487

Epoch [9/20], Loss: 1.1359
Epoch [9/20], Loss: 1.1359
Accuracy: 0.6591375770020534
Classification Report:

	precision	recall	f1-score	support
0	0.85	0.78	0.81	81
1	0.49	0.77	0.60	100
2	0.62	0.41	0.50	82
3	0.82	0.85	0.83	100
4	0.63	0.62	0.62	97
5	0.67	0.07	0.13	27
accuracy			0.66	487
macro avg	0.68	0.58	0.58	487
weighted avg	0.68	0.66	0.65	487

Accuracy: 0.6591375770020534
Classification Report:

	precision	recall	f1-score	support
0	0.85	0.78	0.81	81
1	0.49	0.77	0.60	100
2	0.62	0.41	0.50	82
3	0.82	0.85	0.83	100

4	0.63	0.62	0.62	97
5	0.67	0.07	0.13	27

accuracy			0.66	487
macro avg	0.68	0.58	0.58	487
weighted avg	0.68	0.66	0.65	487

Epoch [10/20], Loss: 1.1338
Epoch [10/20], Loss: 1.1338
Accuracy: 0.6714579055441479
Classification Report:

	precision	recall	f1-score	support
0	0.88	0.79	0.83	81
1	0.54	0.75	0.63	100
2	0.64	0.39	0.48	82
3	0.74	0.90	0.81	100
4	0.63	0.64	0.63	97
5	0.80	0.15	0.25	27

accuracy			0.67	487
macro avg	0.70	0.60	0.61	487
weighted avg	0.69	0.67	0.66	487

Accuracy: 0.6714579055441479
Classification Report:

	precision	recall	f1-score	support
0	0.88	0.79	0.83	81
1	0.54	0.75	0.63	100
2	0.64	0.39	0.48	82
3	0.74	0.90	0.81	100
4	0.63	0.64	0.63	97
5	0.80	0.15	0.25	27

accuracy			0.67	487
macro avg	0.70	0.60	0.61	487
weighted avg	0.69	0.67	0.66	487

Epoch [11/20], Loss: 1.0999
Epoch [11/20], Loss: 1.0999
Accuracy: 0.6652977412731006
Classification Report:

	precision	recall	f1-score	support
0	0.85	0.74	0.79	81
1	0.59	0.69	0.64	100
2	0.64	0.39	0.48	82
3	0.64	0.95	0.76	100
4	0.66	0.66	0.66	97
5	1.00	0.15	0.26	27

accuracy			0.67	487
macro avg	0.73	0.60	0.60	487
weighted avg	0.69	0.67	0.65	487

Accuracy: 0.6652977412731006

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.74	0.79	81
1	0.59	0.69	0.64	100
2	0.64	0.39	0.48	82
3	0.64	0.95	0.76	100
4	0.66	0.66	0.66	97
5	1.00	0.15	0.26	27
accuracy			0.67	487
macro avg	0.73	0.60	0.60	487
weighted avg	0.69	0.67	0.65	487

Epoch [12/20], Loss: 1.0743

Epoch [12/20], Loss: 1.0743

Accuracy: 0.6776180698151951

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.80	0.82	81
1	0.56	0.75	0.64	100
2	0.60	0.43	0.50	82
3	0.74	0.90	0.81	100
4	0.67	0.63	0.65	97
5	1.00	0.15	0.26	27
accuracy			0.68	487
macro avg	0.74	0.61	0.61	487
weighted avg	0.70	0.68	0.66	487

Accuracy: 0.6776180698151951

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.80	0.82	81
1	0.56	0.75	0.64	100
2	0.60	0.43	0.50	82
3	0.74	0.90	0.81	100
4	0.67	0.63	0.65	97
5	1.00	0.15	0.26	27
accuracy			0.68	487
macro avg	0.74	0.61	0.61	487
weighted avg	0.70	0.68	0.66	487

Epoch [13/20], Loss: 1.0707

Epoch [13/20], Loss: 1.0707

Accuracy: 0.6714579055441479

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.84	0.82	81
1	0.57	0.69	0.63	100
2	0.55	0.39	0.46	82

3	0.75	0.86	0.80	100
4	0.63	0.69	0.66	97
5	1.00	0.19	0.31	27
accuracy			0.67	487
macro avg	0.72	0.61	0.61	487
weighted avg	0.68	0.67	0.66	487

Accuracy: 0.6714579055441479

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.84	0.82	81
1	0.57	0.69	0.63	100
2	0.55	0.39	0.46	82
3	0.75	0.86	0.80	100
4	0.63	0.69	0.66	97
5	1.00	0.19	0.31	27
accuracy			0.67	487
macro avg	0.72	0.61	0.61	487
weighted avg	0.68	0.67	0.66	487

Epoch [14/20], Loss: 1.0523

Epoch [14/20], Loss: 1.0523

Accuracy: 0.6735112936344969

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.83	0.83	81
1	0.53	0.80	0.63	100
2	0.54	0.48	0.51	82
3	0.93	0.75	0.83	100
4	0.65	0.66	0.65	97
5	1.00	0.11	0.20	27
accuracy			0.67	487
macro avg	0.75	0.60	0.61	487
weighted avg	0.71	0.67	0.67	487

Accuracy: 0.6735112936344969

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.83	0.83	81
1	0.53	0.80	0.63	100
2	0.54	0.48	0.51	82
3	0.93	0.75	0.83	100
4	0.65	0.66	0.65	97
5	1.00	0.11	0.20	27
accuracy			0.67	487
macro avg	0.75	0.60	0.61	487
weighted avg	0.71	0.67	0.67	487

Epoch [15/20], Loss: 1.0527

Epoch [15/20], Loss: 1.0527

Accuracy: 0.6837782340862423

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.78	0.84	81
1	0.65	0.62	0.64	100
2	0.64	0.46	0.54	82
3	0.62	0.96	0.76	100
4	0.66	0.69	0.67	97
5	0.88	0.26	0.40	27
accuracy			0.68	487
macro avg	0.73	0.63	0.64	487
weighted avg	0.70	0.68	0.67	487

Accuracy: 0.6837782340862423

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.78	0.84	81
1	0.65	0.62	0.64	100
2	0.64	0.46	0.54	82
3	0.62	0.96	0.76	100
4	0.66	0.69	0.67	97
5	0.88	0.26	0.40	27
accuracy			0.68	487
macro avg	0.73	0.63	0.64	487
weighted avg	0.70	0.68	0.67	487

Epoch [16/20], Loss: 1.0180

Epoch [16/20], Loss: 1.0180

Accuracy: 0.6919917864476386

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.83	0.85	81
1	0.66	0.67	0.67	100
2	0.67	0.40	0.50	82
3	0.63	0.96	0.76	100
4	0.68	0.67	0.67	97
5	0.82	0.33	0.47	27
accuracy			0.69	487
macro avg	0.72	0.64	0.65	487
weighted avg	0.70	0.69	0.68	487

Accuracy: 0.6919917864476386

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.83	0.85	81
1	0.66	0.67	0.67	100
2	0.67	0.40	0.50	82
3	0.63	0.96	0.76	100

4	0.68	0.67	0.67	97
5	0.82	0.33	0.47	27
accuracy			0.69	487
macro avg	0.72	0.64	0.65	487
weighted avg	0.70	0.69	0.68	487

Epoch [17/20], Loss: 1.0007
Epoch [17/20], Loss: 1.0007
Accuracy: 0.7125256673511293
Classification Report:

	precision	recall	f1-score	support
0	0.93	0.78	0.85	81
1	0.59	0.83	0.69	100
2	0.67	0.49	0.56	82
3	0.70	0.94	0.80	100
4	0.77	0.60	0.67	97
5	1.00	0.33	0.50	27
accuracy			0.71	487
macro avg	0.78	0.66	0.68	487
weighted avg	0.74	0.71	0.70	487

Accuracy: 0.7125256673511293
Classification Report:

	precision	recall	f1-score	support
0	0.93	0.78	0.85	81
1	0.59	0.83	0.69	100
2	0.67	0.49	0.56	82
3	0.70	0.94	0.80	100
4	0.77	0.60	0.67	97
5	1.00	0.33	0.50	27
accuracy			0.71	487
macro avg	0.78	0.66	0.68	487
weighted avg	0.74	0.71	0.70	487

Epoch [18/20], Loss: 0.9872
Epoch [18/20], Loss: 0.9872
Accuracy: 0.6817248459958932
Classification Report:

	precision	recall	f1-score	support
0	0.88	0.83	0.85	81
1	0.52	0.80	0.63	100
2	0.53	0.51	0.52	82
3	0.94	0.75	0.83	100
4	0.69	0.67	0.68	97
5	1.00	0.11	0.20	27
accuracy			0.68	487
macro avg	0.76	0.61	0.62	487
weighted avg	0.73	0.68	0.68	487

Accuracy: 0.6817248459958932

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.83	0.85	81
1	0.52	0.80	0.63	100
2	0.53	0.51	0.52	82
3	0.94	0.75	0.83	100
4	0.69	0.67	0.68	97
5	1.00	0.11	0.20	27
accuracy			0.68	487
macro avg	0.76	0.61	0.62	487
weighted avg	0.73	0.68	0.68	487

Epoch [19/20], Loss: 0.9903

Epoch [19/20], Loss: 0.9903

Accuracy: 0.702258726899384

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.84	0.86	81
1	0.58	0.76	0.66	100
2	0.60	0.52	0.56	82
3	0.84	0.81	0.82	100
4	0.64	0.69	0.67	97
5	1.00	0.26	0.41	27
accuracy			0.70	487
macro avg	0.76	0.65	0.66	487
weighted avg	0.72	0.70	0.70	487

Accuracy: 0.702258726899384

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.84	0.86	81
1	0.58	0.76	0.66	100
2	0.60	0.52	0.56	82
3	0.84	0.81	0.82	100
4	0.64	0.69	0.67	97
5	1.00	0.26	0.41	27
accuracy			0.70	487
macro avg	0.76	0.65	0.66	487
weighted avg	0.72	0.70	0.70	487

Epoch [20/20], Loss: 0.9726

Epoch [20/20], Loss: 0.9726

Accuracy: 0.704312114989733

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.79	0.83	81
1	0.62	0.71	0.66	100
2	0.65	0.50	0.57	82

3	0.69	0.95	0.80	100
4	0.70	0.65	0.67	97
5	1.00	0.33	0.50	27
accuracy			0.70	487
macro avg	0.76	0.66	0.67	487
weighted avg	0.72	0.70	0.70	487

Accuracy: 0.704312114989733

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.79	0.83	81
1	0.62	0.71	0.66	100
2	0.65	0.50	0.57	82
3	0.69	0.95	0.80	100
4	0.70	0.65	0.67	97
5	1.00	0.33	0.50	27
accuracy			0.70	487
macro avg	0.76	0.66	0.67	487
weighted avg	0.72	0.70	0.70	487

This code loads and preprocesses image data, then splits it into training and validation sets. It converts the data into PyTorch tensors and creates data loaders for batching. The model is initialized on GPU if available, trained on the training set, and evaluated on the validation set. Finally, the trained model is saved to a file.

```
In [12]: y_pred = []
y_true = []

model.eval()

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs = inputs.to(device)

        outputs = model(inputs)

        _, predicted = torch.max(outputs, 1)
        y_pred.extend(predicted.cpu().numpy())

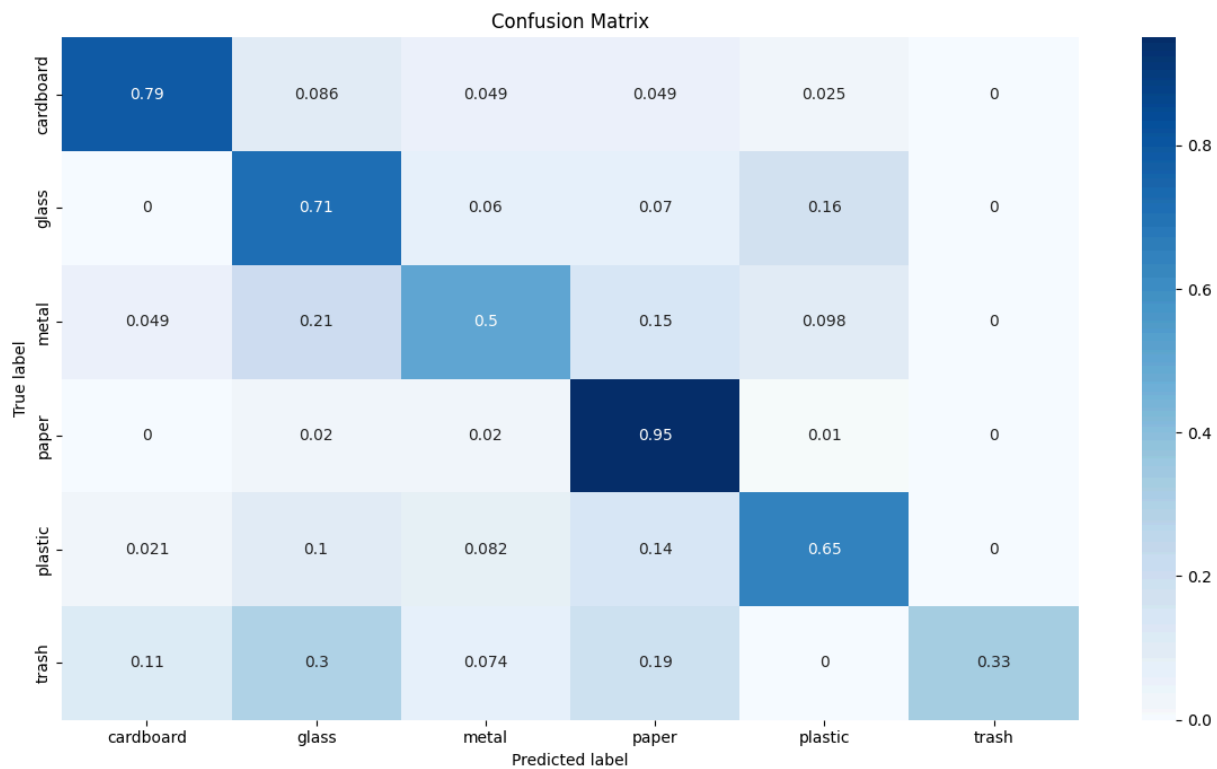
        labels = labels.numpy()
        y_true.extend(labels)

cf_matrix = confusion_matrix(y_true, y_pred)
df_cm = pd.DataFrame(cf_matrix / np.sum(cf_matrix, axis=1)[:], None,
                    index=[categories[i] for i in range(len(categories))],
                    columns=[categories[i] for i in range(len(categories))])

plt.figure(figsize=(12, 7))
sn.heatmap(df_cm, annot=True, cmap='Blues')
```

```
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
plt.show()
```

```
accuracy = accuracy_score(y_true, y_pred)
print(f"Overall Accuracy: {accuracy:.4f}")
```



Overall Accuracy: 0.7043